# Workload Management: DIRAC and DiracX as compute backends

**Alexandre F. Boyer** · CERN

12th DIRAC(X) Users' Workshop · FZU Prague · 13–16 October 2026



# ARM and GPUs are gaining ground




**276 / 500**
TOP500 systems with GPUs, June 2026 (110 in 2018)

**35**
ARM systems in the TOP500 (5 in 2020)

**11k ARM slots**
in ATLAS production at 5 grid sites; up to half of a pledge can be ARM

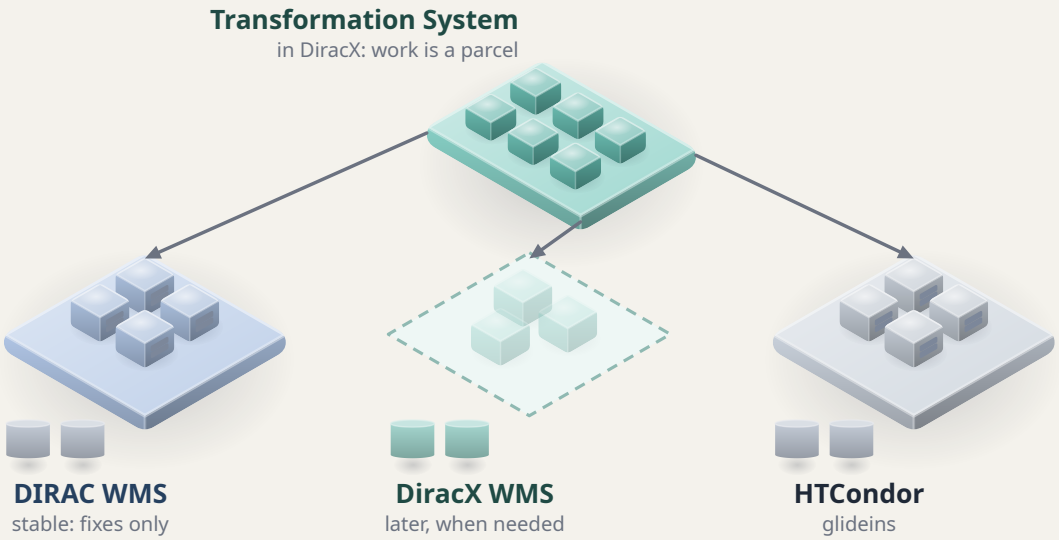
**In DIRAC: temporary solutions**, to keep running on these resources without changing DIRAC, and keep it stable.

**In DiracX: the long-term solutions**, designed for heterogeneous resources from the start.

# 1 · Starting from the transformation system

What it changes for workload management

# The transformation system sees compute backends




*each backend keeps its own data and its own data model*

The transformation system (Tuesday's talks) sends parcels to compute backends. **The DIRAC WMS is one of them**, with its own data, as it is.

# What separate backends give us

### A stable DIRAC

The DIRAC WMS keeps its data and its behaviour. No large change: fixes and performance only, until communities move away from it.

### A DiracX free of the legacy model

DiracX does not inherit DIRAC's jobs, tables or states. Work is a **parcel**; the WMS behind it is designed for it.

### No rush for a DiracX WMS

Most communities can rely on their existing WMS for now: DIRAC, or their own (see the CMS talk just before).

> The DiracX WMS is not a priority. It will move slowly, in parallel, and get a plan when the transformation system needs a native backend.

# 2 · DIRAC: fixes and performance

What keeps it running while we migrate

# Faster

| MEASURED ON PRODUCTION-SIZED DATABASES | BEFORE | AFTER | RELEASE |
|---|---|---|---|
| Task queue matching query | ~30 ms | **~3 ms** | v9.1.0 · #8462 |
| Finding matching task queues (323 queues) | 3.4 s | **0.2 s** | v9.1.20 · #8801 |
| Job insertion | ~115 jobs/s | **~190 jobs/s** | v9.1.20 · #8801 |
| Duplicate task queues during a burst | 64 | **1** | v9.1.20 · #8801 |
| Delay between matches, per job type | 1 s or more | **sub-second** | v9.1.16 · #8711 |

- Also: missing indices on pilots and jobs (#8463, manual `ALTER TABLE`), hot paths found with py-spy (#8524), fewer calls in the pilot agents (#8569, #8624).

> Several fixes made the DIRAC WMS faster and more reliable, for installations at v9 and above. Most of the numbers above need v9.1.

# More reliable

| WHAT WENT WRONG | WHAT HAPPENS NOW | RELEASE |
|---|---|---|
| During bursts of submissions, a task queue could lose its site, platform or tag restrictions. | The restrictions stay; one task queue instead of dozens of duplicates. | v9.1.20 · #8801 |
| The SiteDirector kept sending pilots to banned CEs. | Banned CEs are skipped. | v9.0.2 · #8340 |
| The JobAgent failed jobs that had already been rescheduled. | Those jobs are left alone. | v9.0.17 · #8441 |
| The Watchdog could record NaN or infinite values as job parameters. | Such values are rejected. | v9.1.16 · #8750 |

> None of these change how DIRAC works. They remove ways it could fail.

# Safer

| COMPONENT | PROBLEM, NOW FIXED | FIXED IN |
|---|---|---|
| Job state updates | A user could change the state of someone else's job. | v9.1.20 · v9.0.26 · v8.0.85 |
| Sandbox store | A crafted upload could write files outside the sandbox area. | v9.1.20 · v9.0.26 · v8.0.85 |
| Pilot logging | A crafted request could read files on the server. | v9.1.20 · v9.0.26 · v8.0.85 |
| Pilot manager | SQL injection and missing access checks. | v9.1.10 · v9.0.22 |
| Pilot download | Pilot code fetched over HTTPS without verifying the server. | v9.1.10 · v9.0.22 |
| All WMS databases | Queries built from strings; now parameterised. | v9.1.13 to v9.1.15 |

→ *DIRAC is patched, not developed:* **upgrading is how you get these**

# Heterogeneous nodes: architectures, limits, HPC

**aarch64 beside x86_64** (DIRAC #8456, pull request open) DUW11

- The pilot detects the node's architecture and resolves the container image under the CVMFS `.multiarch` layout: `<base>/<arch>/<image>`.
- Used by the Singularity CE and `dirac-apptainer-exec`.
- Relies on **CVMFS 2.14** (DUCC 2.14.1) on the publishing side to build the layout from multi-architecture images; `unpacked.cern.ch` has it. Any client reads it.
- Left to do: certify, review, merge.

**CPU and memory limits** DUW11

- cgroup2 limits for jobs in containers (#8110).
- Memory taken into account when matching and when filling a multi-core slot (#8366, #8414).

**HPC access** DUW11

- The SSH computing element now uses Fabric (#7703, v9.0).

**Not solved in DIRAC**: a job that can run on x86_64 *or* aarch64 has a single set of requirements.

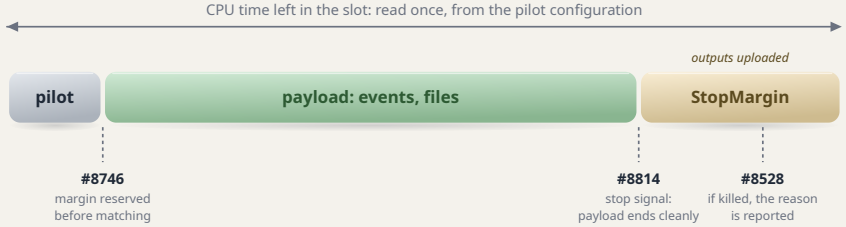
# Time left in the slot

### The problem

- Jobs in containers could not ask the batch system for the time left.
- The post-processing margin was known only to the Watchdog, which could only kill.

**An example**: an MC job sized for 183 events produced 150, was killed, and its output was never uploaded.

**Since v9.1.4** (#8479): the pilot reads the time left once; jobs read it from the pilot configuration.

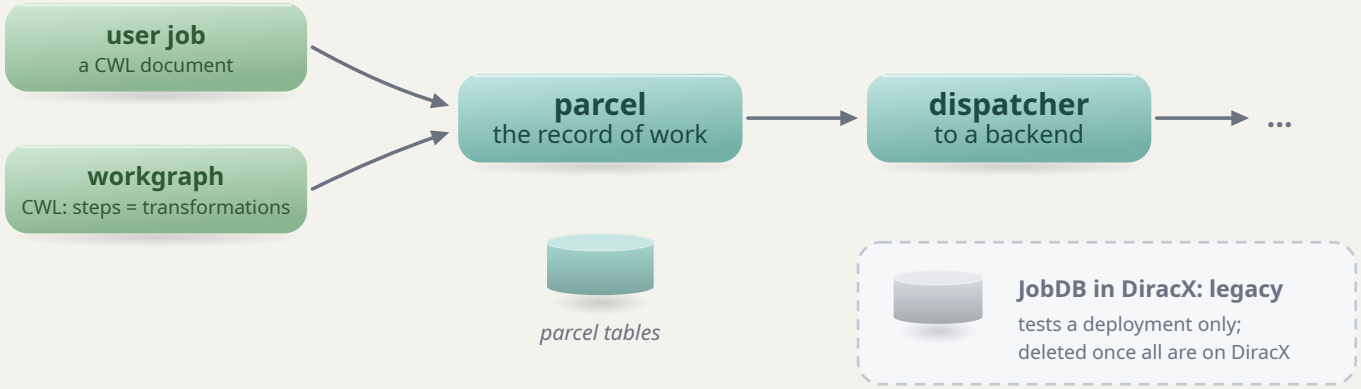



> The three fixes above are in **v9.1.21** only; see "Allocated time management" in the Administrator Guide. CPU work is normalised with DB12: next talk.

# 3 · DiracX: parcels, not jobs

And the proofs of concept around the future WMS

# No jobs in DiracX




- A user job is a parcel, like a bundle of transformation inputs. A failed parcel is final: resubmitting creates a new one.
- A jobs table, if the DiracX WMS needs one, stays private to that backend.
- The JobDB in DiracX today only receives job status updates from DIRAC, to **test that a DiracX deployment works**.
- We do not port job monitoring or the job manager DUW11. **Once everyone is on DiracX, it is deleted.**

# CWL describes all the work

**One format** (DX-ADR-007, draft): a user job, a workgraph and each transformation are CWL v1.2 documents.

- DiracX settings are **hints** under `dirac:`, never requirements: every document stays plain, runnable CWL.
- The same document runs with `cwltool`, locally against grid data, and at scale.
- Processes are resolved before dispatch: the node runs without fetching definitions.

**What this means for a WMS backend**

- Matching requirements come from the hints.
- The job wrapper resolves inputs (`lfn:`, `sandbox:`), captures the output sandbox, and writes a status report per input.
- Outputs are registered by DiracX after the parcel ends, not by the wrapper.

**Today**: `dirac-cwl` 1.3.0; an experimental CWL job wrapper in DIRAC (#8425). The CWL job submission prototype for DiracX DUW11 became part of the transformation system.

# interCEde: one library for computing elements

A typed, asynchronous library to submit to and manage computing elements, usable by DIRAC, DiracX and other workload managers. github.com/DIRACGrid/intercede, proof of concept

## Design

Three ADRs under review (intercede #4): computing elements, integration tests, credentials.

Open points: per-site submit options, whole-node and GPU requests, ARC delegation renewal, cleaning up cloud VMs.

## Code

Core contract proposed this week (#27).

Integration-test scaffolding (#25), and test setups from the communities: HTCondor, ARC with Slurm, Slurm.

## Next

Merge the ADRs, then: core, registry, integration tests, backends.

Last, a synchronous facade and an adapter so DIRAC can use it (#24).

In the backend model, a WMS backend owns the policy (matching, pilots); interCEde owns the mechanics of talking to a CE.

# A matcher for the DiracX WMS

**Goal** (diracx #843): match pilots to work at **1 kHz** (200 Hz at least), with 10 million waiting items per VO. Redis holds an index that can be rebuilt from the database.

**Built**: matching with tag expressions, prioritisation, a benchmark harness, a Redis data model (release 0.4.0, Sep 2026).

**Measured**: the pure Python baseline only, 210 ms per match at 50k candidates, 5.7 s at 750k. No Redis numbers yet.

**What we learned**

- Real requirements vary little: LHCb has 872k waiting jobs but 14 distinct (CPU time, group) pairs. Group work by profile, as task queues do; do not test items one by one.
- An item eligible at ~49 sites must not be stored 49 times.
- Work runnable on two architectures needs a *list* of requirements.

**Next**: matching inside Redis (Lua) and its benchmark on realistic data (#873), then scale tests (#874). **Then an ADR**, written from those numbers, for the matcher of the DiracX WMS backend.

# Pilots: the re-thinking has not started

### Landed

- A pilot database and router in DiracX (#570, Aug 2026) DUW11.
- Pilot: Python 2 removed DUW11; maintenance fixes.

### Paused

- Pilot registration and authentication with pilot tokens (#421).
- Pilot logging through DiracX (#550).
- Untouched since mid-2025.

### Questions for later

- What does a pilot need to know about parcels?
- What belongs to the pilot, what to the job wrapper?
- One model for pull and push?

> Pilots belong to the DiracX WMS backend, which comes after the transformation system. We continue to patch the DIRAC pilot.

# The job wrapper: one for all of us, owners wanted

### Today there are several

- DIRAC's JobWrapper: pre-processing, payload, post-processing; an offline variant for push.
- The `dirac-cwl` wrapper, experimental in DIRAC.
- Executor prototypes in DiracX.

Each community also adds its own steps around the payload.

### Requirements to gather

- Inputs: data, sandboxes, protocol access.
- Time left, graceful stop, containers, architectures.
- Nodes without outbound network.
- Reporting: status, parameters, logs.
- What happens when an upload fails.

**We are looking for owners**: people who collect the requirements from the communities, set priorities, and accept the result, in a product-owner role. Come and talk to us this week.

# Take-aways

### THE MODEL

- The transformation system sends parcels to compute backends; the DIRAC WMS is one of them.
- DIRAC stays stable; DiracX is free of DIRAC's job model.

### FOR INSTALLATIONS

- Upgrade to v9.1: faster matching and job insertion, time-left fixes (v9.1.21), security patches.
- Multi-architecture images: #8456, soon.

### FOR DIRACX

- No jobs: parcels. The JobDB in DiracX only tests deployments, then goes.
- The DiracX WMS is not a priority; it will get a plan when needed.
- CWL describes everything that runs.
- Matcher: Redis numbers, then an ADR. interCEde: ADRs close to merge. Pilots: not started.

### WE NEED YOU

- Owners for the job wrapper.

# Thank you

Questions and comments welcome, this week and after.

**Alexandre F. Boyer** · alexandre.boyer@cern.ch

github.com/DIRACGrid · diracx.diracgrid.org

